Computer Networks 248 (2024) 110476

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

A framework for detecting zero-day exploits in network flows

Almamy Touré [a,b,*], Youcef Imine [a], Alexis Semnont [b], Thierry Delot [a], Antoine Gallais [a]

[a] Univ. Polytechnique Hauts-de-France, INSA Hauts-de-France, LAMIH, CNRS, UMR 8201, F-59313 Valenciennes, France
[b] IBM Security, France

ARTICLE INFO

Keywords:
Zero-day
Intrusion detection system
Machine learning
Deep learning
Cyber attack

ABSTRACT

Zero-day attack detection solutions aim to proactively identify unknown threats targeting valuable assets within a given system. While many Intrusion Detection System (IDS) solutions leverage learning techniques to build novel attack detection systems, they often focus on enhancing accuracy for specific attack types, overlooking the potential for multiple attack scenarios. Therefore, we introduce a novel framework for detecting zero-day attacks that evade current detection systems. Our framework enhances attack identification and qualification through a hybrid learning approach, where supervised learning ensures detection of known attacks and unsupervised learning. It encompasses intrusion detection phases from data collection to new attack class detection by identifying anomalies in real-time network flow data. Unsupervised learning, which involves grouping similar data points into clusters, establishes minimum distances within these clusters. This process triggers cluster division when certain thresholds are reached. Finally, an online supervised learning process validates our approach's effectiveness in identifying anomalies associated with zero-day attack flows.

This approach significantly reduces the False Detection Rate (FDR) without solely focusing on optimizing machine learning (ML) and deep learning (DL) algorithms hyper-parameters. We evaluated our framework on two datasets: one from a real industrial context at IBM and the NSL-KDD dataset. The results demonstrate our framework's ability to detect anomalies in previously zero-day attack targets. On average, we identified 71 anomalous flows per target, achieving an overall average online learning accuracy of 98.4% for the IBM dataset and 96.6% for the NSL-KDD dataset, thereby validating the detection of these new attack scenarios.

1. Introduction

According to the IBM 2023 Data Breach Report, the average cost of a data breach in 2023 was 4.45 *milliondollars*, marking a 15% increase over three years [1]. These figures highlight the growing impact of cyber threats on organizations worldwide. Among these threats, unresolved known vulnerabilities significantly impact businesses. Additionally, zero-day attacks emerge as particularly formidable adversaries, exploiting vulnerabilities unknown to security experts and existing defense mechanisms [2–4]. In an ever-evolving landscape of cyber threats, the challenge of safeguarding information systems remains constant.

Intrusion Detections Systems play a critical role in identifying and mitigating these elusive threats. The complexity of these threats necessitates innovative approaches that extend traditional security measures.

However, existing methodologies for detecting known and zero-day attacks often fall short. Traditional signature-based systems struggle to identify new threats lacking predefined signatures, and new signature detection solutions are limited in their ability to cover various attack categories. Additionally, anomaly detection techniques face challenges in distinguishing malicious anomalies from legitimate ones, often resulting in a high rate of false positives [5–9].

In response to these challenges, recent years have seen the emergence of innovative detection techniques leveraging ML [10,11] and DL algorithms [12,13]. These approaches offer the ability to discern abnormal patterns and behaviors within data streams, thus providing more sophisticated methods for zero-day detection. However, they are also confronted with limitations such as dependence on representative training data, difficulty in generalizing to new threats, improvement of precision without hyper-parameter's optimization, and model complexity, hindering their real-time deployment on resource-limited systems [9,14].

To address these limitations, this research presents a new zero-day detection approach designed to overcome the shortcomings of existing methods. Building upon previous research [15], our solution offers a comprehensive framework integrating supervised and unsupervised learning techniques, algorithmic fusion, and anomaly detection. Thus, we aim to enhance the effectiveness and efficiency of zero-day attack detection in cyber systems and cover any new attack families identifiable by network flow collectors.

* Corresponding author at: Univ. Polytechnique Hauts-de-France, INSA Hauts-de-France, LAMIH, CNRS, UMR 8201, F-59313 Valenciennes, France.
E-mail address: almamytoure27@gmail.com (A. Touré).

https://doi.org/10.1016/j.comnet.2024.110476
Received 9 January 2024; Received in revised form 7 March 2024; Accepted 30 April 2024
Available online 13 May 2024
1389-1286/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

Through this work, we aim to contribute to the advancement of zero-day attack detection and pave the way for future research in this critical domain. Thus, our contributions to this zero-day detection framework compared to existing works in the field revolve around the following key points:

- **Hybridization of classification techniques:** Integrating supervised and unsupervised classification techniques is essential. The utilization of supervised classification models such as CNN aids in categorizing known network flows, while employing unsupervised algorithms like K-Means unveils concealed patterns in the data.
- **Precision optimization through combined learning methods:** The approach emphasizes algorithms fusion, combining the prowess of supervised CNN-based methods with decision trees (DT), random forests (RF), K-Nearest-Neighbors (KNN) and Naive Bayes (NB). This cross-breeding aims to maximize the detection model's precision, offering a more accurate and dependable insight into potential threats, without solely focusing on hyper-parameter's optimization.
- **Correlation between supervised and unsupervised results:** Creating a correlation table between supervised classification outcomes and clusters formed by the unsupervised approach aids in associating clusters with attack classes or normal flow categories.
- **Outlier-based anomaly identification for zero-day detection:** Identifying outliers within clusters formed by the K-Means algorithm and using this data to create new potential attack classes enables proactive zero-day attack detection. This facilitates real-time model updates through online learning, adapting the model regularly to new data, thereby enhancing its capability to detect and respond to zero-day attacks in real-time.
- **Validation of results on two datasets:** One derived from an industrial context containing real flows and recent attacks, and the other from the constantly used state-of-the-art dataset for intrusion detection and zero-day detection mechanism validation.

In the following sections, we delve into various aspects of intrusion detection. Section 2 delves into a comprehensive analysis of existing solutions to detect both known and zero-day attacks, highlighting their limitations. Section 3 provides a detailed context by describing the feature engineering of network flows and introduces the concepts of zero-day attacks and online learning. Our framework for detecting zero-day attacks, proficient in identifying both known and zero-day attacks, is presented in Section 4. Section 5 focuses on a theoretical analysis of our anomaly identification methodology, based on silhouette score formulas. Transitioning to Section 6, we address datasets and apply our framework to detect zero-day attacks. Section 7 encompasses discussions on our work and the obtained results, laying the groundwork for future research in this domain. Finally, Section 8 summarizes our work and paves the way for future advancements in zero-day attack detection.

2. Related works

IDS serve the purpose of identifying malicious activities that may compromise a Host (HIDS) or a Network (NIDS) [16]. Distinguishing these activities from legitimate flows requires solutions able to classify the network traffic. These solutions (see Table 1) need to be effective not only in detecting known, referenced attacks but also in identifying zero-day attacks, which, according to IBM Security, occur when a malicious actor exploits a zero-day vulnerability to implant malware, steal data, or otherwise cause damage to users, organizations, or systems [17].

The initial solutions aimed to classify network traffic by comparing the port number in a packet to the port number assigned by IANA, analyzing the content of the transmitted data and identifying specific applications. This was done by extracting signature information from the payload of the application layer and comparing it to signatures within an established KB. These Traditional approaches [18–21], relying on port, payload, or signature-based detection, fall short due to obfuscation techniques such as encryption, steganography, tunneling, anonymization, mutation, morphing, and physical obfuscation. These methods enhance data protection and pose challenges to incident detection via network flow collectors [22]. Consequently, leveraging automatic and DL techniques becomes crucial for intrusion detection.

Various known attack detection solutions and techniques for identifying zero-day attacks have been explored.

2.1. Detection techniques for known attacks

In [10], Jabbar et al. introduced a classification solution using alternating DT, an algorithm that creates prediction and separation nodes. This approach, applied to the NSL-KDD dataset, inspired further research aiming to optimize hyper-parameters and enhance accuracy rates while exploring different algorithms. Additionally, in [11], the authors employed the NB algorithm and its implementation with PCA on the NSL-KDD dataset. Similarly, in [23], Koc et al. applied the Hidden NB (HNB) to the KDD Cup 99 dataset, combining attribute independence with a binary HNB classifier to reduce naivety. Adversarial learning approaches increasingly challenge ML-based detection systems. Techniques to evade ML-based attack detectors using the RF algorithm have been proposed, highlighting the sensitivity of ML algorithms, particularly RF, to parameter changes and feature order [24].

DL, extending ML principles, processes large datasets using hidden layers for in-depth learning [25]. As with ML algorithms, most of the attack detection approaches proposed with DL models focus on increasing accuracy and reducing false positives. In [12], Zhao et al. proposed a sequential classifier using Artificial Neural Network (ANN) to decrease false positives over the KDD99 dataset. Similarly, in [13] Vinayakumar et al. implemented a multi-layer approach for complex feature extraction, aiming to enhance ANN efficiency in NIDS. However, varying accuracies across datasets (KDDCup 99 and UNSW-NB15) highlight the necessity for more universally applicable solutions. Efforts in attack detection research encompass aspects such as reducing execution times, preventing model overlearning, and optimizing hyper-parameters. For instance, in [26], Sivamohan et al. combined RF algorithms with PCA for feature extraction from the CICIDS2017 dataset and utilized Bi-directional LSTM (BiLSTM) for attack detection, accelerating computations with GPUs. Similarly, in [27], Wang et al. employed Convolutional Neural Networks (CNN) and LSTM for spatial and temporal feature extraction, albeit experiencing decreased accuracy for unbalanced data. Azizjon et al. used a CNN and LSTM combination to serialize TCP/IP packets for supervised learning on the UNSW-NB15 IDS dataset, utilizing CUDA GPU acceleration to reduce computational time [28]. In [15], Almamy et al. introduced a CNN-based approach for attack detection. Their method aims to extract common features from all network flows, regardless of the context and specific network. By employing 1D-CNN, they identify additional characteristic patterns within these flows, followed by pooling and flattening techniques to classify network flows. Their model effectively distinguishes legitimate from attack flows and detects various attack classes across two distinct datasets.

These solutions facilitate classification, the differentiation between legitimate and malicious flows, and the identification of attack classes within the model. However, they do not ensure zero-day attack detection.

2.2. Detection techniques for zero-day attacks

Building on these capabilities to discern between legitimate and malicious activities, we present methods for detecting zero-day threats.

Table 1
Comparison of zero-day attack detection approaches.

Attack detection approaches	Signature-based	Statistical anomaly	Supervised learning	Unsupervised learning	Hybrid	Our
References	[5–7]	[8,9,29]	[10,11,15,23,24]	[34,37]	[36,38,39]	
Ability to detect zero-day attacks	Low	Medium	Medium	High	High	Very high
Sensitivity to false positives	Medium	High	High	Variable	Variable	Low
Complexity and Resource requirement	Low	Medium	Medium	Variable	Variable	Medium
Adaptability to new attack types	Low	Medium	Medium	High	High	Very high
Ease of deployment in industrial environment	High	Variable	Variable	Variable	Variable	High

Linear transformations aid in simplifying and reducing the dimensionality of data, contributing to improved anomaly detection sensitivity and specificity [29]. This can contribute to improving the sensitivity and specificity of anomaly detection, as the extracted features can better capture differences between normal and abnormal data. A combination of these two techniques, linear transformation and anomaly detection, along with known attack signatures, can be effective in zero-day detection. In [30], different discriminant functions are used to calculate the estimated probability of zero-day attacks by analyzing network connection features. Thus, the detection of zero-day attacks is done by identifying anomalies using a defined anomaly score and the KNN algorithm. The proposed model has been tested on the NSL-KDD dataset.

Some research efforts have relied on auto-encoders for zero-day attack detection. Auto-encoders are neural network architectures used for data compression, dimensionality reduction, and data generation [31–33] . An implication of an auto-encoder is presented in [34]. A high recall rate and a low false negative rate have been defined with mean square error (MSE) thresholds. Additional unsupervised techniques involve local detection modules to aggregate detected anomalies [35]. Botnet detection approaches by Blaise et al. focus on identifying significant changes in port usage, utilizing statistical measures to detect anomalies over time, evaluated on two datasets: MAWI and UCSD.

Hybrid approaches are also proposed in the state of the art. In [36], Pu et al. introduce an intrusion detection method that combines Sub-Space Clustering (SSC) and One Class Support Vector Machine (OCSVM). By integrating subspace clustering techniques, SSC forms clusters from small subspaces of the original dataset, while OCSVM, an extension of SVM, handles unlabeled data to detect anomalies. The authors evaluated their solution using NSL-KDD and compared it to three state-of-the-art unsupervised solutions.

The state-of-the-art in attack detection concentrates on optimizing hyper-parameters, ensuring consistent performance across diverse datasets, and mitigating model overlearning. Challenges persist due to the ever-evolving dynamics of botnets, their adaptability to evade detection, and the need for effective strategies against stealthy attacks. Long-term methodologies may limit effectiveness against rapidly adaptive botnets, swiftly altering traffic patterns.

3. Background

3.1. Flow features engineering

In this section, we highlight certain contributions regarding the features engineering of network flow from our work [15].

The definition and content of a network flow are crucial in intrusion detection. A flow network represents network activity by standardizing IP addresses, ports, byte and packet counts, and other data into flow records, effectively documenting network sessions between two hosts [40]. Understanding the composition of a flow is vital for analyzing network traffic and user activity within an information system.

During this communication session between two elements over a given time interval, packet exchanges allow the extraction of network characteristics, relevant regardless of the context or information system. These characteristics fall into three main categories, namely:

- **Asset characteristics** gather features related to Network, Transport and Application layers of the OSI model.
- **Flow characteristics** regroup features related to the session layer of the OSI model, including the type of session established, the direction and the duration of the communication.
- **Exchanged data and rate characteristics** in which we find features related to exchanged contents (e.g., amount of communicating data).

This feature engineering, based on the definition of a network flow, aims at extracting from the raw data, relevant and useful features, reducing the processing time of a model to feed a network activity classification algorithm. It aims at finding a set of universal minimalist features and facilitating several network flow analysis tasks. Thus, it can be used to classify network activity.

3.2. Zero-day attack

Zero-day attacks pose a significant threat in the realm of cybersecurity. These attacks derive their name from their exploitation of vulnerabilities previously unknown to software developers, the targeted system or network infrastructures. In essence, attackers leverage these vulnerabilities before patches or defense measures are available [14].

In contrast to other types of attacks that exploit known vulnerabilities, zero-day attacks target undisclosed vulnerabilities. Due to their novel nature, zero-day attacks are particularly challenging to detect by both traditional and recent security systems. Zero-day attacks differ from other attack types, such as classic brute force attacks or standard port scanning attacks. Whereas the latter rely on well-known methods and established attack vectors, zero-day attacks are characterized by their unpredictability and ability to bypass conventional defenses.

3.3. Online learning

Online learning, also known as continual or incremental learning, is an ML method where the model is updated as new data becomes available, without the need to retrain the model on the entire dataset [41,42]. In contrast to offline learning, where the model is trained once on a fixed dataset, online learning enables continuous adaptation to changes in data and environments [43]. In the context of zero-day attack detection, online learning can be used to update the detection model in real-time as new attack examples are observed. This approach allows for increased responsiveness to emerging attacks and continuous adaptation to changes in attackers' behavior.

4. Zero-day solution

In our new zero-day detection framework, we propose a comprehensive five-phases approach to address the ever-evolving landscape of security threats related to network flows. These phases are mainly designed to play a critical role in data collection, data classification, data clustering, correlating the input dataset and anomaly detection.

In our approach, we start by centralizing the network flows to be analyzed, creating training and evaluation datasets, establishing a KB of known attack scenarios, and continually feeding this base with new attack scenarios through anomaly detection for the new data the system continues to receive and process. In what follows, we will describe the different phases of our framework.

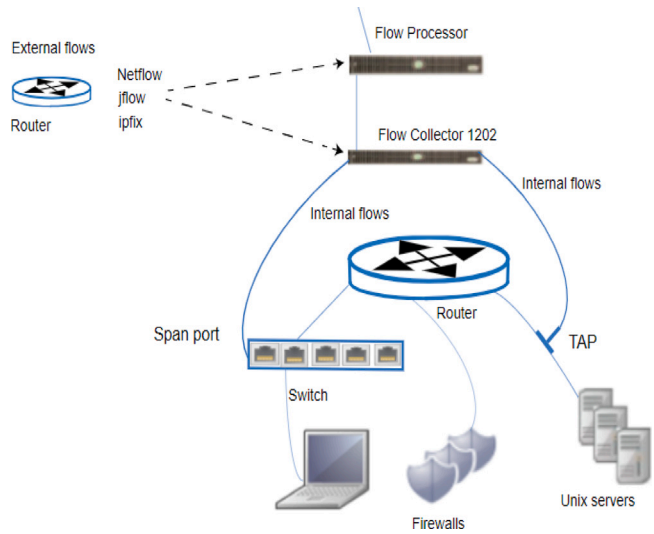


Fig. 1. Data collection.

Table 2
The classification of network flow features into different families.

Flow characteristics	
Flow-ID	Flow type
Flow-aggregation-count	Flow direction
Flow-bias	Flow duration
Asset characteristics	
Source-IP	Source-port
Destination-IP	Destination-port
Protocol	Category
Application	Application-group
Data and speed characteristics	
Source-bytes	Destination-bytes
Bit-per-second	Total bytes

4.1. Data collection

The first phase of our framework involves the collection of data related to the network traffic of the system. This phase requires a flow collector to capture packets passing through the network and its surroundings. The diagram in Fig. 1 illustrates the network flow collection options (internal and external traffic), as proposed by IBM Security's architecture [40].

The collector retrieves flow data from raw packets collected via monitoring and mirroring ports [44] such as SPAN, TAP, sessions supervising, or from external flow sources such as NetFlow [45], sFlow [46], and jFlow [47]. The data from these various internal and external sources are centralized in a dedicated flow collector. Following this, we will apply the feature engineering as presented in Section 3.1.

Then, we will extract multiple pieces of information from the collected data. Consequently, the `input-dataset` will consist of attributes based on the three defined families of characteristics needed to classify network flows. In Table 2, we describe in detail the various feature families.

Once these characteristics are defined, the `input-dataset` is composed by identifying and selecting attack scenarios flows experienced by the information system, along with a set of legitimate flows identified at various periods (morning activities, business hours, evening activities, weekends, and so on). So, the `input-dataset` must be representative of the various data variants collected and centralized within the flow collector.

As depicted in Fig. 2, these meticulously curated data will fuel both supervised and unsupervised models.

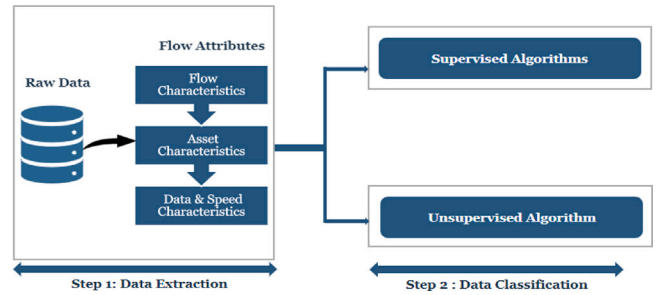


Fig. 2. Our proposed model for data categorization.

4.2. Supervised classification phase

Once the `input-dataset` is formed, the next phase of our framework consists of a supervised classification of network flows. This means associating each flow with the appropriate class (legitimate flow or malicious flow).

We use deep learning-based classification, acquiring specific features during the learning phase. Notably, we leverage the CNN for its feature detection capabilities within the convolutional layer, alongside other DL approaches. This feature detector detects the features that will match the highest values of the `input-dataset` in the feature map. A convolution operation is then performed between the `input-dataset` and the feature detector to determine a feature map that is representative of the relevant data. This model aims at coupling this feature map building process with the feature engineering presented in Section 3, aiming to define a robust method for classifying the network flows.

Thus, the feature map is injected into the CNN's other layers (pooling, flattening and fully connected) in order to determine the output categories known as the `Target-Set`:

- **Target Not Associated with Incident:** which describes a first category of flows not associated with attacks (normal or legitimate traffic).
- **Targets Associated with Incidents:** made up of malicious activities related to different attack scenarios.

We use a One-Dimensional CNN (1D-CNN) approach with parameters for `nb-filters` filters to extract `nb-filters` different features on the first convolution layer of the network.

Once the `Target-Set` is defined, we move to the boosting step to decrease the FDR (false positives, false negatives). To achieve this, the other algorithms used include DT, RF, KNN, and Bernoulli NB. In our model, DT and RF are used because of their effectiveness in classifying nonlinear data, given the highly varied and complex attack patterns of network flows. KNN will enable us to define complex boundaries using its proximity-based method, which is useful as we have a variety of normal flows and attacks.

Finally, the NB model, based on its assumption of conditional naivety, presumes that the features used for prediction are all independent of each other. This implies that the presence or value of one feature does not influence the others. This notion will enable us to evaluate the relevance of the attributes proposed in the `input-dataset`. Thus, the boosting technique, a set-based learning method that combines a set of weak `Learners` (algorithms) into one strong learner, will rely on the four other algorithms.

An implementation of our CNN-based solution and the four other ML algorithms gives us an initial prediction of the different categories of network flows. We rely on the `F1 score` of predictions for each class. Assigning weights to these predictions as shown in Fig. 3, we compare the weighted F1 scores, class by class, from each algorithm to determine an optimal prediction for each label (normal flow and

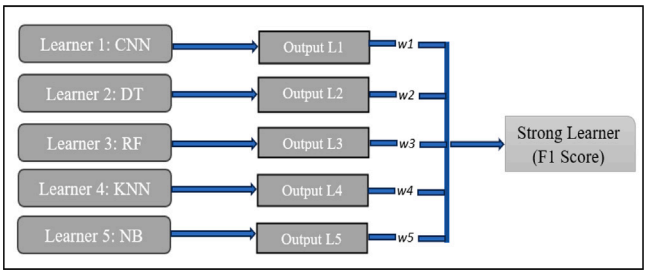


Fig. 3. Boosting technique.

attack flow). The CNN, serving as the foundational algorithm, holds precedence (a higher weight) over the other algorithms. The `Target-Set` will be used to create a correlation Section 4.4, which will serve as the basis for analyzing the formed clusters in the phase 4.3 and for online learning in the phase 4.5. This approach will avoid running the supervised model entirely during real-time data collection and classification.

Depending on the dataset or the context, the optimal hyper-parameters may change, so we will not focus on this aspect of model optimization. Therefore, the four models defined in addition to the CNN are employed with the parameters commonly used.

After completing the supervised classification phase, we transition to the next stage, where we employ unsupervised clustering using the robustness of the K-Means algorithm. This enables the grouping of similar segments within unlabeled data, unveiling concealed patterns within our network data.

4.3. Unsupervised classification phase

In this phase, we employ an unsupervised algorithm to identify data clusters in space, offering a visual representation of the input data. The network flows of data contain various attack classes and have a non-hierarchical structure. The K-Means algorithm is more suitable among unsupervised methods to address the clustering of these data types. Our application of the K-means algorithm adopts an approach where the final cluster centroids are defined randomly in the `input-dataset`. This decision was made to explore the algorithm's adaptability to the diverse network flow data. Indeed, choosing a random centroid configuration ensures a more comprehensive exploration of potential structures within the data, providing an unbiased perspective on the network flow clustering process. Consequently, this model will allow us to divide our dataset into K distinct groups and extract patterns of malicious flows related to attack scenarios, legitimate flows or both. The Elbow method analysis allowed us to identify a point where the precision showed only marginal improvement with the addition of clusters. To enhance anomaly detection and achieve a finer segmentation of the data, we opted to double this point to attain a more detailed distribution, thereby better capturing the data intricacies crucial for anomaly detection.

At the completion of the various implementation phases of this algorithm, the clusters obtained will form a set called the `Cluster-Set`.

There are three types of clusters built, considering that a network flow is either normal or abnormal. Thus, this flow could be positioned within one of the following groups:

- **Only Attack Flows** containing only attack data.
- **Only Normal Flows** clusters with only legitimate flows.
- **Both** are made up of normal traffic and attacks.

Next, we move on to the analysis stage of the formed `Cluster-Set` and essential metrics such as the intra-cluster distance `a(i)`, the inter-cluster distance `b(i)`, the average distances `d-mean`, the maximum distance `d-max`, and most importantly, the silhouette score `S-sil`. All these metrics allow us to determine the minimum distance `d-min` per cluster that `i` belonging to a cluster should not exceed. We employ `outlier` distance analysis to compare this distance threshold for each cluster `d-min` with the new data that the flow collector continues to receive. Indeed, the set of point distances that will exceed this `d-min` are termed `outliers` within a cluster. Subsequently, we will evaluate `Data Collected in real time` to identify potential outliers, thus enhancing our ability to detect and respond to zero-day attacks in real-time.

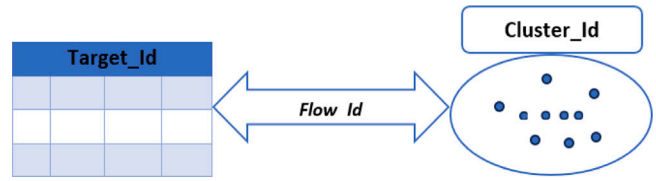


Fig. 4. Correlation with flow Id.

Table 3
An example of a correlation table.

Cluster-id	Target-id
a	25% target1 + 30% target2 + 45% target3
b	5% target4 + 95% target3
c	1% target2 + 8% target0 + 91% target5
d	100% target0

In order to analyze and correlate the `Target-Set` defined at the end of the supervised approach (phase 4.2) and the `Cluster-Set` created in the phase 4.3, we will create a join table of the two sets, called *Correlation Table*.

4.4. Correlation table phase

To analyze and exploit the results of unsupervised learning (K-Means), and also associate them with of the supervised approach, we create the correlation table. This table allows us to detect which output class from the supervised approach best corresponds to which cluster of points from the unsupervised approach. We recall that in our framework, each communication flow has a unique identifier, `Flow-Id`, as presented in Fig. 4. Based on the flow identifiers and associated targets of the supervised approach, we look for the appropriate clusters. Therefore, for each network flow, we identify the target of the supervised approach with the cluster number of the unsupervised approach. Table 3 is an example of correlation table. This association of target and cluster, via the `Flow-Id` enables us to determine the best flow classes for the clusters and also to validate the predictions of the supervised approach. To do this, we determine the similarity ratios between the elements of a cluster and each target.

The selection of the number of clusters K is deliberate to optimize the spatial representation of the data. Following the establishment of similarity ratios between the points within a cluster and each defined target, it is noteworthy that each cluster tends to be primarily associated with a single target, even though we have three types of clusters. And each target is represented by at least one cluster. Once this correlation table is defined, we employ `outlier` distance analysis to identify outliers.

4.5. Outlier detection phase

The distance measure serves to assess the proximity between new data points and existing ones. This is crucial in our framework aimed at identifying zero-day attack scenarios, often associated with distant points within established clusters.

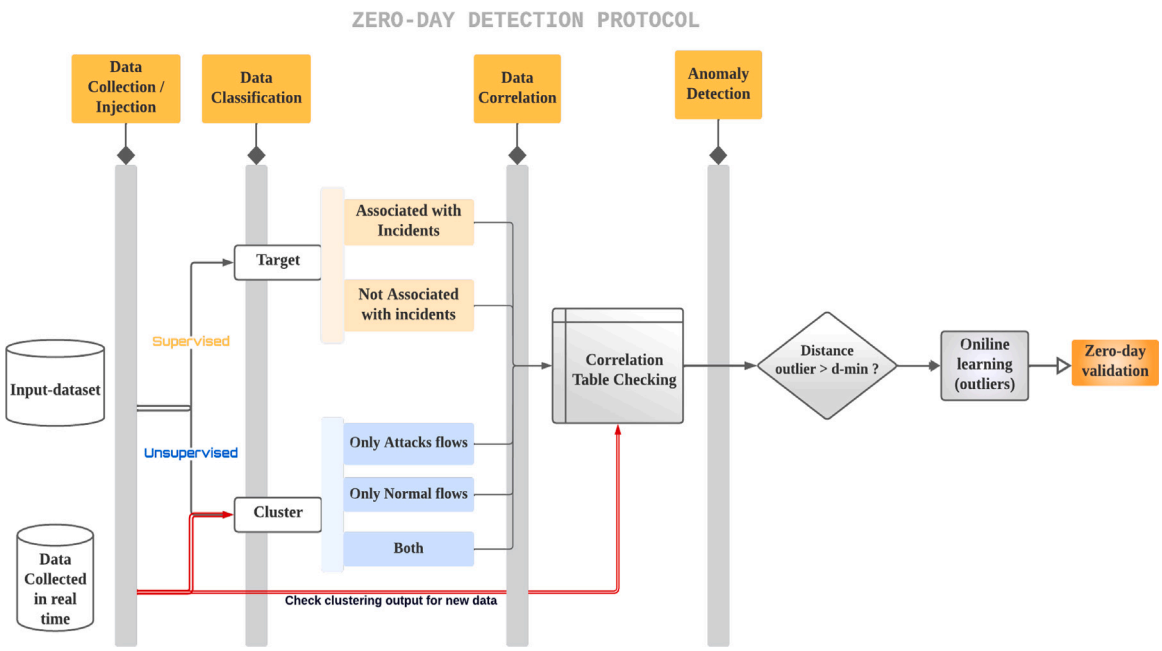


Fig. 5. Our proposed zero-day detection approach.

Our input dataset comprises real-time data collected by the system, as presented in Fig. 5. In this new configuration, the `Data collected in real time` is injected into the unsupervised approach without altering any hyper-parameters or the `Cluster-Set` itself. This decision is driven by the need to detect anomalies caused by abnormal distances within clusters.

To understand the data's structure and potential classes, we leverage the correlation table. Subsequently, we calculate the distance `d(i, ol)` between the positioning of the real-time data and the existing cluster elements for classification. As a reminder, any data point significantly distant from others in its assigned cluster is identified as an `outlier`. Determining whether to split a cluster depends on the calculated `d-min` and the presence of new points as outliers with distances exceeding the `d-min`.

Next, the online learning is applied to adapt the model to the remaining data points that are not outliers. Indeed, a zero-day attack class does not solely consist of outliers. It can also encompass data that does not reach the d-min threshold in terms of average distance. This learning approach enables our model to be regularly updated as new data becomes available. In that way, it will be adapted in real time to the information provided by the information system [48]. Thus, the outliers in a cluster whose distance respects the defined `d-min` will constitute the training data for a new attack class. The other new points in the same cluster that are not outliers are used as evaluation data for the overall model. This last evaluation is used to match the points of the new outlier clusters and validate the detection of zero-day attacks. To do this, we use the same learning algorithms: CNN and the four boosting algorithms described in Section 4.

We define `deg-th` the degradation threshold established for base models within the context of online supervised learning. This threshold indicates the acceptable level of performance degradation of the base models in terms of overall model accuracy.

The online learning model is retained if it remains below this degradation threshold, ensuring its stability and efficacy. However, if the base model experiences degradation beyond this threshold, it is not preserved or retained, indicating a significant deterioration in its performance.

Fig. 5 highlights the various phases of our framework outlined in Section 4, from data collection to zero-day attack identification.

5. Theoretical analysis

As we have seen in Section 4.5, the outlier detection in our framework is decided based on whether the distance between the outlier and the cluster centroid exceeds a distance $d-min$. This section will present a method to determine this distance on the basis of the Silhouette score.

First, let us start by presenting some fundamental principles allowing the optimization of intra-cluster coherence and inter-cluster dissimilarity in the K-means clustering algorithms:

- **The cluster center** is the average of all the features of the points belonging to that cluster. Let **Ci** be the center of cluster C, **Ni** be the number of points in cluster C, and **xij** be the coordinates of the jth point in cluster C:

$$C_i = \frac{1}{N_i}\sum_{j=1}^{N_i} x_{ij}$$

- **The Silhouette score** which quantifies the separation of clusters.
  1. For an individual data point i:

$$S_{sil}(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

  where a(i) and b(i) represent the intra-cluster distance and the inter-cluster distances respectively for a given point i.
  2. For a cluster C:

$$S(C) = \frac{1}{|C|}\sum_{i \in C} S_{sil}(i)$$

Building upon the above definitions, we aim to ascertain the relevance of maintaining the outlier points within a specific cluster or not. To do so, we start by identifying the various scenarios for adding new points within a given cluster C.

- **Scenario 1:** The new elements added to the cluster C are very close the old elements of the cluster in terms of distance. Therefore, they do not significantly impact the intra-cluster and inter-cluster distances, causing, at worst, a minor degradation of the silhouette score. In our framework, we treat this scenario as one case where there is no evidence of detecting a new attack scenario. Therefore, we consider these new elements either as normal traffic or as a known attack pattern.

– **Scenario 2:** all the new elements, added to cluster C, are relatively far from the old elements of the cluster. Therefore, adding these relatively distant elements may (depending on their number) impact the intra-cluster distance, causing a significant degradation of the silhouette score. In our framework, we consider this degradation as an indicator of a new attack, which implies a restructuring of these root clusters.
– **Scenario 3:** at least one element added to the cluster C is far from the old elements in terms of distance, while others blend in with the existing elements. This scenario is common in evolving corporate networks due to the use of various business applications and employees not always rigorously adhering to the Information Security Management System (ISMS). Furthermore, the exploitation of zero-day vulnerabilities can also contribute to the occurrence of anomalous and responsive flows in this scenario.

In what follows, we will present an analysis of the distance based on scenario 3, where we consider that we have at least one new element in the cluster that is far from the old ones. Our analysis will study whether it is better to split the cluster or not if scenario 2 occurs. The results of our analysis can, however, be generalized to scenario 2, where all the elements added to the cluster are far from the old ones.

Case 1: Maintaining the cluster is the best solution.

In the case where the addition of a new outlier element ol in the cluster C does not imply the creation of a new cluster, we will mainly observe a change in the intra-cluster distance $a(i)$. Therefore, by adding one element ol to the cluster, the silhouette score will be represented as follows:

$$S(C) = \frac{1}{|C|}\sum_{i=1}^{|C-1|}\frac{b(i)-a'(i)}{\max\{a'(i),b(i)\}} + S_{sil}(ol) \tag{1}$$

with a new intra-cluster score $a'(i)$

$$a'(i) = \frac{|C-1|\cdot a(i) + d(i,ol)}{|C|} \tag{2}$$

We note that $|C|$ represents the cardinality of after adding one outlier, $a(i)$ represents the intra-cluster score before adding ol and $d(i,ol)$ is the distance between an old element i and the new element ol.

$$S(C) = \frac{1}{|C|^2}\sum_{i=1}^{|C-1|}\frac{b(i)\cdot(|C|)-a(i)\cdot|C-1|}{\max\{a'(i),b(i)\}} + \frac{d(i,ol)}{\max\{a'(i),b(i)\}} + S_{sil}(ol) \tag{3}$$

From the resulting formula (3) above, we define the function `f1(X)` of the variable X relative to the distance `d(i,ol)` and $S_{sil}(ol)$. This function represents the variation in the silhouette score of the cluster $S(C)$.

With

$$\max\{a'(i),b(i)\} = b(i) \tag{4}$$

$$f1(X) = S(C) = \frac{1}{|C|^2}\sum_{i=1}^{|C-1|}\frac{b(i)\cdot(|C|)-a(i)\cdot|C-1|}{b(i)} + \frac{d(i,ol)}{b(i)} + S_{sil}(ol) \tag{5}$$

Given (4), we draw X the variable in the formula (5), dependent on both the distance between each point i and the identified outlier, as well as the silhouette score of this outlier. This is how we obtain the formula (6).

$$f1(X) = S(C) = \frac{1}{|C|^2}\sum_{i=1}^{|C-1|}\frac{b(i)\cdot(|C|)-a(i)\cdot|C-1|}{b(i)} + X \tag{6}$$

When

$$max\{a'(i),b(i)\} = a'(i) \tag{7}$$

$$f1(X) = S(C) = \frac{1}{|C|^2}\sum_{i=1}^{|C-1|}\frac{b(i)\cdot(|C|)-a(i)\cdot|C-1|}{a'(i)} + \frac{d(i,ol)}{a'(i)} + S_{sil}(ol) \tag{8}$$

Based on (7), we get the variable X in formula (8).

$$f1(X) = S(C) = \frac{1}{|C|^2}\sum_{i=1}^{|C-1|}\frac{b(i)\cdot(|C|)-a(i)\cdot|C-1|}{a'(i)} + X \tag{9}$$

Case 2: Cluster splitting is the best solution.

This implies that the distance of the outlier significantly affects the silhouette score $S(C)$. Therefore, it is necessary to create a new neighboring cluster. In the case of cluster splitting, a variation is observable at the cluster inter-distance `b(i)`.

Continuously, based on the silhouette score formulas presented in 5, we will get the silhouette score when the outlier significantly impacts the cohesion of the original cluster.

$$S(C) = \frac{1}{|C|}\sum_{i=1}^{|C|}\frac{d(i)-a(i)}{\max\{a(i),d(i,\mathrm{ol})\}} \tag{10}$$

In this case, with the cluster splitting into two, the inter-cluster distance of the original cluster becomes the average distance `d(i,ol)` between different points `ii` and the outlier. Hence, Formula (3) is obtained. Function `f2(X)` is defined to assess the variation of the silhouette score in this second scenario of original cluster separation, based on the distance `d(i,ol) = X`.

With

$$\max\{a(i),d(i,\mathrm{ol})\} = d(i,\mathrm{ol}) = X \tag{11}$$

When the inter-distance $b(i)$ represents the maximum between $a(i)$ and $b(i)$, as defined in formula (11), it results in the following form for `f2(X)` presented in the formula (12).

$$f2(X) = S(C) = \frac{1}{|C|}\sum_{i=1}^{|C|}1 - \frac{a(i)}{X} \tag{12}$$

When

$$\max\{a(i),d(i,\mathrm{ol})\} = a(i) \tag{13}$$

And in the case where the maximum between the intra-distance and the inter-distance is the intra-distance $a(i)$, as depicted in the formula (13), then the function `f2(X)` appears as follows :

$$f2(X) = S(C) = \frac{1}{|C|}\sum_{i=1}^{|C|}\frac{X}{a(i)} - 1 \tag{14}$$

Once both functions are defined, we will subsequently determine the minimum distance from which this cluster separation is appropriate. This `d-min` corresponds to the point of intersection between the two functions f1(X) and f2(X). Any distance `X` greater than this intersection point is considered an `outlier` of the original cluster and is used as the training basis for online learning, as described in Section 4.

6. Evaluation - results

In this section, we will assess the effectiveness of our zero-day attack detection framework while also emphasizing improved detection of known attacks within the collected network flows, aiming to minimize the FDR. To do this, we will apply the various phases of our framework, as described in Section 4, to two datasets: the IBM dataset extracted from a real industrial context and the NSL-KDD dataset from the literature. Therefore, we identify and collect network data associated with these two contexts. Then, we apply feature engineering coupled with the CNN-based model to these data, and use other algorithms for the boosting technique, in order to constitute the different classes of normal

Table 4
IBM Dataset content validated with MITRE ATT&CK.

Flow-class	Target-ID	Tactic	Technique	Size (#rows)
Normal	0	–	–	24 854
Large-leakage	1	TA0010	T1567	4253
Stealthy-leakage	1	TA0010	T1030	4254
Critical indicator of compromise	7	TA0001	T1189	2064
Traffic to anonymization service	2	TA0011	T1090	288
Traffic to a phishing domain	5	TA0001	T1566	759
High inbound emails from external host	4	TA0011	T1071	481
Access to a certificate expired service	3	TA0042	T1588	305
Remote flood (TCP)	6	TA0040	T1498	21

and attack flows. Next, we use an unsupervised approach based on the K-Means algorithm to classify the data in space and thus determine the data clusters. A correlation table is used to link classes of supervised flows and clusters of unsupervised flows. Finally, we analyze the notion of distance for new data injected into our framework and identify zero-day attacks to the model. We validate this anomaly detection using an online learning process, relying on supervised classification and its algorithms.

We define different evaluation cases for our framework that vary according to the evaluation data injected.

6.1. Evaluation settings

Our architecture was implemented using Python within the `Jupyter Notebook` environment, supported by a machine equipped with an `Intel® Core™ i5-1135G7 11th generation processor` and 16 GB of RAM, running on a `64-bit Windows 11` operating system. For modeling and learning tasks, we relied on `scikit-learn` for ML algorithms, `NumPy` and `Pandas` for data manipulation, `Matplotlib` and `Seaborn` for result visualization. `TensorFlow` was utilized specifically for implementing neural networks.

6.2. Framework phase 1: Leveraging the IBM dataset

IBM QRadar is a security appliance that is built on Linux. QRadar allows for the collection, storage and correlation logs for the detection of security incidents. To prove the effectiveness of our approach, we extracted raw data from a real-time industrial context, with an IBM proprietary IDS probe (QRadar Network Insight, QNI), which extends QRadar by providing a detailed view of real-time network communications [49].

Our dataset includes data collected in 2023 and, thus, recent attack chains. The flows used in our classification have been extracted over a period of one week. This extraction contains legitimate flows and non-legitimate ones that we categorize using the framework MITRE ATT&CK [50]. This content is presented in Table 4. To evaluate our zero-day framework, we will use two attack classes, the first with ID 6 and the second with ID 7.

For preprocessing and normalization, we perform transformations on our raw data to facilitate its processing by the learning algorithms we employ. Firstly, we do a clean-up by not keeping any data containing null values so as not to introduce bias into the model. The raw data has an average of 150 features by default. However, using all these features for classification is not an optimal approach in terms of execution time. Therefore, in our solution, we reduce the number of features to 14 (at most) according to our feature engineering approach (Section 3) in addition to one target that describes the flow class.

After this extraction, all decimal entries (e.g., rate, amount of data, source/destination port numbers) remain unchanged. On the other hand, each byte in a field related to an `IP address` is converted to its hexadecimal value (excluding the dots that separate the bytes). After that, these values are concatenated, which gives us a unique value that will be converted to a decimal. The field related to the `flow direction` has at most four possible values in the raw data; those relating to the `flow bias` have at most five possible values, and the type of communication protocol used is TCP or UDP. We use *OneHotEncoder* on these few non-numerical features to create a binary vector (composed of 0s and 1s) for each possible category or class in the categorical variable. Although the field values are now all numerical, they have very different ranges, so scaling or normalization is needed to make the data comparable and bring them down to a common scale. We have therefore carried out various normalizations on the data: standardization, minmax, tanh, lognormal. The different experiments performed have shown that normalization with Standardization allows us to obtain the best results later on, because we have better data separation, which is very useful in our case.

6.3. Results

To determine the quality of our classification model and to evaluate its performance against various existing learning algorithms, we used the following four popular evaluation metrics:

$$Accuracy = \frac{(TP + TN)}{(FP + FN + TP + TN)},\ Precision = \frac{TP}{TP + FP},$$
$$Recall = \frac{TP}{TP + FN},$$
$$ScoreF1 = 2 * \frac{(Precision * Recall)}{(Precision + Recall)}$$

where, for a given flow class C: True Positive (TP) represents the number of flows correctly classified in the given class C; True Negative (TN) represents the number of flows correctly classified outside of class C; False Positive (FP) represents the number of flows wrongly classified in the class C; and finally False Negative (FN) represents the number of flows wrongly classified outside the class C.

Secondly, the silhouette score S_{sil} of a cluster and a set of clusters presented in Section 5 is used to assess the quality of the clusters defined, to determine the minimum distance from an outlier necessary to split a cluster, and therefore to identify a zero-day.

Our input-dataset consists of eight classes. We will identify attack classes that will be used to evaluate the zero-day detection capability of our framework. In the upcoming steps, we exclude the attack class that we define as the zero-day to be detected. This attack class will represent our `Data collected in real time`. Thus, instead of eight classes, the model initially processes seven classes: a first class of legitimate flows with ID 0, and seven classes of attack flows with IDs from 1 to 7. Targets 6 and 7 represent `Data collected in real time` that we will detect using the framework.

6.3.1. Framework phase 2: Building the Target-Set

In the initial stage of evaluating our zero-day detection framework, we begin by forming the `Target-Set`. To achieve this, we apply a multi-class classification to our dataset (Table 4), defined after the feature engineering, the preprocessing and the normalization.

As a reminder, the F1 score represents an arithmetic average of precision and recall. It serves as a metric to measure the ability of learning algorithms to effectively associate flows with legitimate traffic classes or their corresponding target attacks.

Table 5
The multi-class classification with our CNN model.

Target	Our CNN model (zero-day 6)				Our CNN model (zero-day 7)			
	Precision	Recall	F1-score	Support	Precision	Recall	F1-score	Support
0	0.97	1	0.98	4491	0.99	0.99	0.99	4466
1	1.00	1.00	1.00	2798	1	1	1	2857
2	0.86	0.94	0.90	64	0.86	0.96	0.91	51
3	0	0	0	69	0.75	0.81	0.78	54
4	0.95	0.65	0.77	150	0.97	0.69	0.80	159
5	1.00	1.00	1.00	95	1	1	1	89
6	x		x	x	1	1	1	7
7	0.99	0.99	0.99	420	x	x	x	x
Accuracy		98%				99%		

Table 6
Classification with boosting algorithms for zero-day 6.

T	DT			RF			BernoulliNB			KNN		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	1	1	1	1	1	1	0.98	0.98	0.98	0.99	0.99	0.99
1	1	1	1	1	1	1	1	1	1	1	1	1
2	0.98	0.97	0.98	1	1	1	0.73	1	0.84	0.94	0.98	0.96
3	1	0.99	0.99	1	0.99	0.99	0.59	0.97	0.74	0.88	0.97	0.92
4	0.98	0.99	0.98	0.99	0.97	0.98	0.91	0.13	0.23	0.89	0.82	0.85
5	1	1	1	1	1	1	1	1	1	1	1	1
7	1	1	1	1	1	1	0.95	1	0.97	0.98	0.99	0.99
Acc		100%			100%			97%			99%	

Table 7
Classification with boosting algorithms for zero-day 7.

T	DT			RF			BernoulliNB			KNN		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
0	1	1	1	1	1	1	0.97	0.98	0.98	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1
2	1	1	1	1	1	1	0.71	1	0.83	0.91	0.96	0.93
3	0.98	1	0.99	1	0.98	0.99	0.57	0.96	0.72	0.93	0.94	0.94
4	0.97	0.98	0.98	0.99	0.99	0.99	0.97	0.21	0.35	0.94	0.89	0.92
5	1	1	1	1	1	1	1	1	1	1	1	1
6	1	1	1	1	1	1	1	1	1	1	1	1
Acc		100%			100%			97%			99%	

Table 8
Clustering scores for zero-day 6 and 7.

Top 5 cluster - zero day 6					Top 5 cluster - zero day 7				
Cluster	Score Sil	a(i)	b(i)	Count	Cluster	Score Sil	a(i)	b(i)	Count
C4	0.766	0.448	2.058	3698	C7	0.781	0.466	2.297	3740
C12	0.528	0.650	1.403	2386	C10	0.316	1.168	1.754	2543
C1	0.281	1.100	1.659	2350	C3	0.487	0.697	1.393	2484
C35	0.337	1.094	1.699	2289	C30	0.296	1.195	1.746	2441
C0	0.364	0.958	1.479	2065	C22	0.326	1.536	2.346	2263

The `input-dataset` is directly fed into the CNN algorithm, which forms the basis for detecting known attacks, as described in Phase 4.3 of Section 4.

As observed in Table 5, the CNN yields an average F1 score of 0.49 for target 3, representing network traffic with an expired digital certificate. The low precision in this class is attributed to the neural network's operation, and the selected generic features lack information about the validity or expiration of a certificate. Consequently, the convolution layers struggle to effectively detect this attack class. A lower F1 score is noted for target 4, with an average of 0.78, indicating potential external phishing emails. The neural networks seem to be confused between these two attack targets. On a positive note, the results for the other targets are very promising, with F1 scores ranging between 0.90 and 1 in both executions and an overall precision ranging from 98% to 99%.

For effective detection of our zero-day attacks, we require improved detection of known attacks, implying a low FDR. Indeed, given the low F1 scores of targets 3 and 4, and that of target 2, we will combine the results of the CNN algorithm with four supervised ML algorithms through a boosting technique based on the F1 score.

As observed in Tables 6 and 7, the results demonstrate that all the algorithms have achieved exceptional performances, with 100% accuracy for DT and RF. This suggests the remarkable ability of the chosen models to generalize well and correctly classify each category. Bernoulli NB exhibits slightly lower performance, with an overall precision of 97%. This is attributed to the probabilistic nature of the algorithm, which assumes conditional independence between features, an assumption that may not fully align with the complex relationships present in the data. As for KNN, it also displays excellent performance with an overall precision of 99%, showcasing its ability to adapt well to the data structure by utilizing the spatial proximity of instances.

The boosting technique aims to leverage different classification algorithms to reduce mispredictions and capture the specificities of each class, which can vary from one algorithm to another. Combining the results from Tables 5–7, we achieve a maximum F1 score of 1 for targets 0, 1, 2, 5, and 6, and a score of 0.99 for the remaining targets

Table 9
The correlation table for the evaluation data.

Data with ZD 6		Data with ZD 7	
Cluster-Id	Target-Id	Cluster-Id	Target-Id
31	0	12	4
40	1	31	1

3 and 4. These classification results ensure a better distinction between legitimate flows and attacks, leading to the effective detection of known attacks with negligible FDR.

6.3.2. Framework phase 3: Cluster-Set building

In the second phase of our framework, we will form the `Cluster-Set`. To achieve this, we apply the unsupervised K-Means algorithm to our `input-dataset`, with two executions corresponding to each zero-day attack.

For both scenarios, we obtain a cluster count of 25 using the Elbow method. The S_{sil} is 0.53 when the zero-day is the `Critical Indicator of Compromise` attack and 0.54 when the zero-day is the *DOS* attack. In an effort to better utilize space and cluster learning, the cluster count is doubled from 25 to 50 in both scenarios. Therefore, our final K value is 50.

In Table 8, we present the top 5 clusters with the highest number of entries `Count`. Subsequently, we determine the intra-cluster distances `a(i)` and inter-cluster distances `b(i)`, from which the silhouette score `Score Sil` of each cluster in our unsupervised learning is calculated. These results highlight the data's structure.

The silhouette score provides an initial indication of the clustering quality. Clusters C4 (ZD-6) and C7 (ZD-7) stand out with high scores (0.766 and 0.781, respectively), indicating a clear separation of elements within the clusters and a distinct demarcation between clusters. These results suggest a well-defined intrinsic structure within the data.

Furthermore, the intra-cluster and inter-cluster distances provide additional insights into the internal cohesion of clusters and their external separation. Despite having similar silhouette scores, clusters C4 and C7 exhibit distinct characteristics. C4 (ZD-6) displays lower internal cohesion a(i) but a more pronounced external separation b(i), while C7 (ZD-7) shows higher internal cohesion and equivalent external separation. These nuances reflect the complexity of the underlying data structures.

The size of clusters, reflected by the number of elements they contain, is a crucial aspect. Clusters C4 and C7 stand out not only for their metric performance, but also due to their significant size (3698 and 3740 elements, respectively). This indicates a strong representation of these clusters within their respective datasets. These values per cluster subsequently allow us to calculate the maximum distance `d-min` from which it is necessary to decide whether to separate a cluster or not, as defined in the theoretical analysis (Section 5).

We recall that function `f1(X)` represents the variation in the silhouette score of the original cluster when new elements are retained

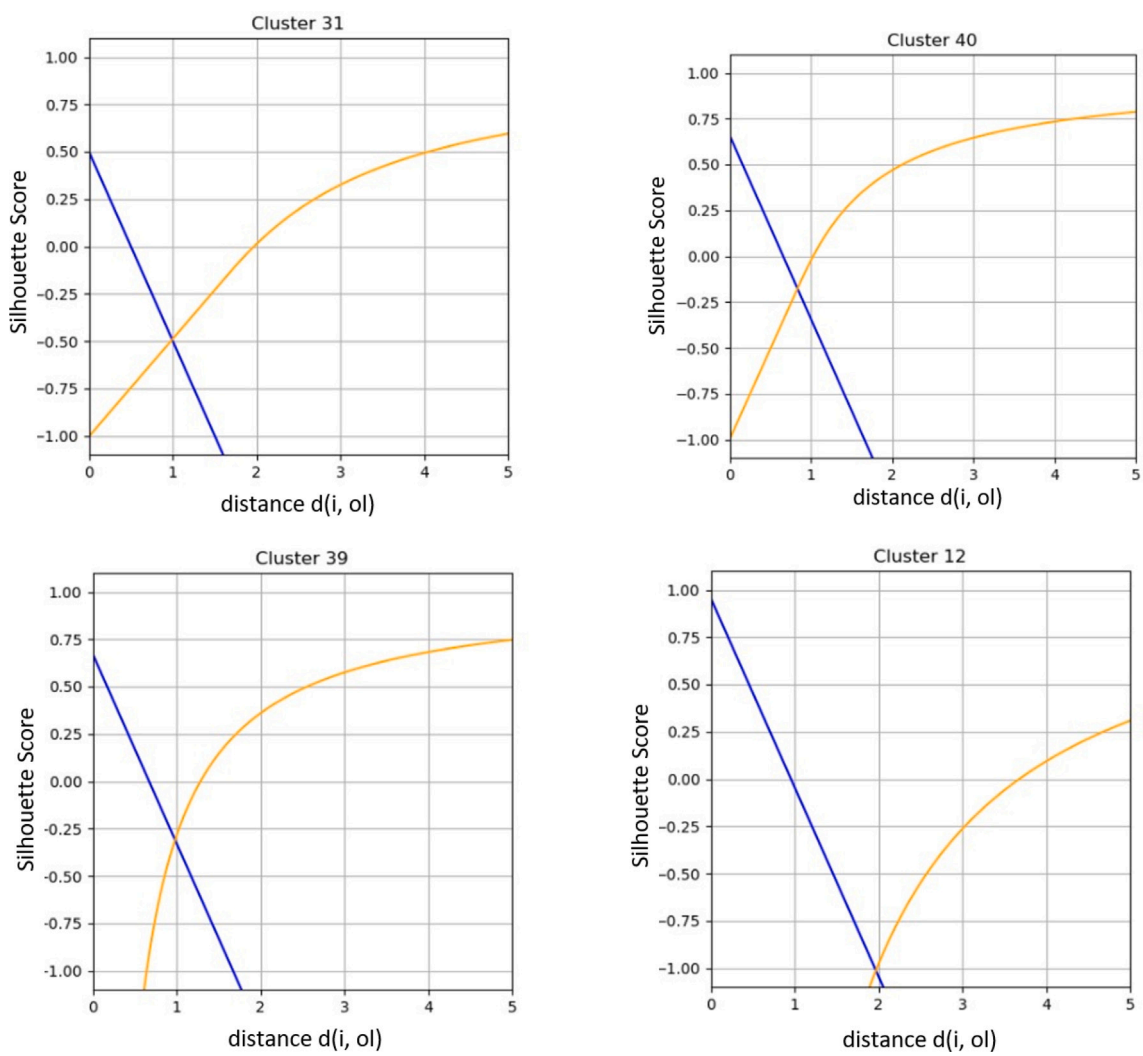


Fig. 6. Determining the minimum distance (d-min) based on the cluster scores calculated from the functions f1 and f2 for the IBM dataset.

Table 10
Data distribution for zero-day 6.

Zero-day 6	Cluster-ID 31			Cluster-ID 40		
	Before	Outliers only	All data	Before	Outliers only	All data
Count	184	3	187	1556	18	1574
Mean	1.45	153.67	3.89	0.75	42.03	1.22
Max	4.46	170.45	170.45	2.52	87.25	87.25
d-min	1	132,6	x	0.85	3.81	x
Score Sil	0.45	x	0.44	0.45	x	0.44
Online learning	YES			YES		

within it, while function `f2(X)` describes the variation when the original cluster must be split. Fig. 6 illustrates an example of identifying the minimal distance d-min (the intersection point of the two functions `f1(X)` in orange and `f1(X)` in blue) between clusters 31 and 40 for zero-day 6, and clusters 12 and 32 for zero-day 7.

6.3.3. Framework phase 4: Correlation table

In Table 9, we display the clusters in which the two evaluation data were positioned and also the targets corresponding to these clusters.

Now, we can proceed to detect zero-day attacks by injecting new data or streams that have never been learned by the models in the previous phases. Two classes of attacks have been defined, and these will be tested one by one.

6.3.4. Framework phase 5: Distance analysis for detecting zero-day

As described in the architecture of the zero-day framework, the new data to be evaluated are injected into the unsupervised approach without any variation in the parameters of this model. Therefore, we do not modify the number of clusters or the hyper-parameters of the K-Means algorithm.

In this phase of the framework, we analyze the placement of the new data for evaluation, the clusters in which they will be positioned, the distance measure of these data concerning the previous points in the target clusters, and most importantly, the variation in the silhouette score of each cluster with the addition of new points. This variation is crucial in deciding whether to retain these elements in the cluster or choose to split it.

Use Case1: Zero-day Attack Target ID 6 It involves DOS data, defined as an attack class never previously learned by our model. Therefore, technically, this class represents a zero-day for our model. The DOS data has been injected into the K-Means clustering, and all the points have been placed into two different clusters: C31 and C40.

As we observe in Table 10, before the introduction of zero-day data, the identified clusters (C31 and C40) had distinct characteristics, with notable differences in the number of elements, averages, and maximum distances. C31 contained 184 elements with an average of 1.45 and a maximum distance of 4.46, while C40 had 1556 elements with an average of 0.75 and a maximum distance of 2.52. The addition of DOS data significantly impacted the structure of the clusters. The number of elements increased for C31 (rising to 187) and C40 (reaching, 1574). Furthermore, the values of averages and maximum distances underwent significant changes. For C31, the average increased to 3.89 and the maximum distance to 170.45, whereas for C40, the average rose to 1.22 and the maximum distance to 87.25. The silhouette score, prior to the addition of zero-day data, for both clusters C31 and C40 decreased from 0.45 to 0.44.

These results suggest that the addition of zero-day data has altered the configuration of existing clusters, increasing the dissimilarity between the newly introduced data and the data already present.

The *Online learning* field, based on theoretical analysis of distance, and the variation in silhouette score, confirms the transition to the online learning stage when its value is set to YES. Table 11 showcases the results from online learning of various classification algorithms on the `input-datatset`, including the identified outliers within the clusters. In these findings, the label F1 score corresponds to a split on the zero-day data. Hence, predictions are made on zero-days that are either *outliers* and the threshold value for acceptable degradation of the base models, denoted as `deg-th`, has been set at 0.005.

Table 11
Online learning for zero-day 6.

Model	DT	RF	KNN	NB	Our CNN
Normal score	1.0	1.0	0.99	0.97	0.98
Online score	0.998	0.999	0.994	0.975	0.986
Label F1	1.0	1.0	1.0	1.0	1.0
Model to kept	YES	YES	YES	YES	YES

As observed in Table 11, all models remain below this threshold after online training, affirming their stability and justifying the retention of all models for zero-day attack detection. Therefore, the outliers represent zero-day flows, effectively detected by our framework.

Use Case2: Zero-day Attack Target ID 7

For the second use case, we will utilize the data from the `Critical Indicator of Compromise` class as never-before-learned data and inject it into the model.

As observed in Table 12, prior to the introduction of zero-day data, the identified clusters (C31 and C40) displayed notable differences in the number of elements, means, and maximum distances. C31 contained 184 elements with a mean of 1.45 and a maximum distance of 4.46, while C40 comprised 1556 elements with a mean of 0.75 and a maximum distance of 2.52. The addition of D0S data significantly impacted the cluster structure because the number of elements increased for C31 (rising to 187) and C40 (reaching, 1574). Moreover, both the mean and maximum distances underwent substantial changes. For C31, the mean escalated to 3.89, and the maximum distance surged to 170.45, while for C40, the mean rose to 1.22 and the maximum distance reached 87.25.

Similar to the first use case, these findings suggest that the introduction of zero-day data altered the configuration of the existing clusters, increasing the dissimilarity between the newly introduced data and the pre-existing ones.

In Table 13, it is evident that the outliers do not lead to degradation in the base models. This observation underscores the models' strong ability to generalize well to new online data, particularly zero-day instances.

6.4. Exploring the NSL-KDD dataset and conducting comparative analysis

A functional approach that remains independent of specific datasets, hyper-parameters, and context serves as a reference model. Following the evaluation of our detection framework on an industrial dataset, we intend to leverage a public dataset to test its applicability. Our selection has been directed towards NSL-KDD. This dataset [51] is an enhanced version of the KDDCup99 dataset [52], which is a standard dataset consisting of a wide variety of simulated intrusions in a military network environment, defined in 1998 by MIT Lincoln Labs of the U.S. DARPA agency. NSL KDD has a KDDTrain+ set (125,973 records) divided into 22 types of attacks and normal traffic and a KDDTest+ test set (22,544 records), which contains 14 additional attacks not included in the KDDTrain+. The set of attack classes is divided into four main

Table 12
Distribution for zero-day 7.

Zero-day 7	Cluster-ID 12			Cluster-ID 39		
	Before	Outliers only	All data	Before	Outliers only	All data
Count	7	1	8	179	98	277
Mean	2.42	8.21	3.14	0.89	6.61	2.91
Max	2.79	8.21	8.21	6.15	7.23	7.23
d-min	1.96	6.79	x	0.99	6.37	x
Score Sil	0.44	x	0.42	0.44	x	0.42
Online learning	YES			YES		

Table 13
Online learning for zero-day 7.

Model	DT	RF	KNN	NB	Our CNN
Normal score	1.0	1.0	0.97	0.972	0.99
Online score	0.998	0.999	0.993	0.974	0.982
Label F1	1.0	1.0	0.96	0.72	0.96
Model to kept	YES	YES	YES	YES	YES

families: Denial of Service (DOS), Remote To Local (R2L), User To Root (U2R), Probe. The content of NSL-KDD is presented in Table 14.

In this dataset comprising 41 features, we apply the detailed feature engineering outlined in Section 3 to retain only the features: `duration`, `protocol-type`, `service`, `land`, `src-bytes`, `dst-bytes`, `count`, `srv-count`, `same-srv rate`, `srv-diff-host-rate`, `wrong-fragment` and `flag`.

The NSL-KDD dataset exhibits inconsistencies primarily in data variability, where characteristics between the training and test sets differ significantly, potentially impacting model generalization. Additionally, variations in attack definitions between training and test sets may lead to inconsistent evaluation results. Given these inconsistencies between the test and training data in the NSL-KDD dataset, we decide to solely focus on the training data. From this, we exclude a few subclasses of each attack family: neptune, satan, perl, spy, phf. Similar to the approach used in industrial datasets, we chose a subclass from the DOS family as a zero-day attack, specifically the teardrop with 892 entries. Additionally, 5000 random entries from the normal class are included. This constitutes the input dataset. Subsequently, data normalization is performed to proceed with the classification steps (using a 70% 30% train-test split for supervised classification) as outlined in our framework.

As evidenced by Table 15, we present the overall results of various boosting algorithms and the base CNN algorithm. The results demonstrate remarkably high accuracy for all algorithms, with scores nearing or reaching 100 in the weighted measure. However, upon closer examination of the macro-avg and weighted-avg measures, there are some noteworthy points to highlight. The DT displays very good overall performance, with precision, recall, and an F1 score of 0.92, 0.98, and 0.93, respectively, for the macro-avg measure, showcasing its strong ability to correctly identify classes. RF shows slightly better performance than DT, with higher scores in precision (0.94), recall (0.97), and F1 score (0.94) for the macro-avg measure. NB exhibits slightly lower precision compared to DT and RF, with an F1 score of 0.80 for the macro-avg measure, indicating challenges in classifying certain classes. The KNN algorithm demonstrates robust performance with F1 scores of 0.95 for the macro-avg measure, showing a similar capability to DT but with a slight decrease in precision. These algorithmic results further solidify those of the CNN, which demonstrates high performance, with F1 scores of 0.99 for the macro-avg measure.

Once the `Target-set` for this dataset is established, we present in Table 16 only the variation in metrics of the cluster impacted by the addition of zero-day elements: C0. Our zero-day data corresponds to the teardrop subclass of the DOS family. The data from this subclass has been placed within cluster C0, with 94 outliers identified concerning the d-min distance of this cluster, as shown in the Fig. 7.

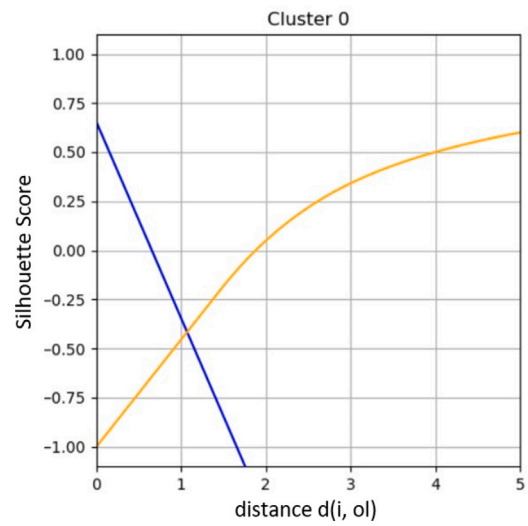


Fig. 7. Determining the minimum distance (d-min) based on the cluster scores calculated from the functions f1 and f2 for the NSL-KDD dataset.

In Table 16, it is evident that the minimum outlier distance, d(i,ol), is significantly smaller than the theoretical `d-min` distance of cluster C0. Additionally, the inclusion of outliers within C0 notably degrades the silhouette score, decreasing from 0.651 to 0.428, considering the weight of 94 outliers.

This degradation is further supported by the average distance within the new concatenated set of old elements and outliers, which increases from 1.455 to 2.966, more than doubling. This necessitates transitioning to the phase of online learning (Online learning = YES) to validate the detection of a new attack class that the model has not encountered before.

Based on the results from Table 17, online learning highlights the detection of a zero-day, as no model degrades significantly compared to the defined threshold. However, the KNN model is not retained because the F1 score for the new zero-day class is very low, standing at 0.739.

Therefore, having completed all the steps of our framework, we can conclude that the teardrop class, previously defined as a zero-day, has been successfully detected as a new class of anomalies, never previously learned by our model.

In Table 18, we provide a comparative analysis between the performance of our framework and that of several state-of-the-art solutions for zero-day attack detection. We highlight key characteristics of the presented solutions, such as the utilized datasets, specific types of targeted zero-day attacks, detection algorithms employed, and the precision rates of the models.

7. Discussion

In line with our ongoing work, several points of discussion can be explored to further enhance the detection and response to known and zero-day threats:

Table 14
The NSL-KDD dataset content.

Flow-class	Subclass	Tactics	Techniques	Train (#rows)	Test (#rows)
Normal	Normal(67343)	–	–	67 343	9711
DOS	Back(956), Land(18), Neptune(41214), Pod(201), Smurfs(2646), Teardrop (892)	TA0040	T1498	45 927	7458
R2L	Guess-Passwd(53), ftp-write(8), Imap(11), Phf(4), Multihop(7), Warezmaster(20), Waresclient(890), Spy(2)	TA0001	T1133/T1199	995	2754
U2R	Buffer-overflow(30), Loadmodule(9), Perl(3), Rootkit(10)	TA0004	T1548	52	200
Probe	Satan(3633), Lpsweeo(3599), Nmap(1493), Postsweep(2931)	TA0043	T1589/T1590	11 656	2421

Table 15
Classification with supervised algorithms.

Model	DT			RF			BernoulliNB			KNN			Our CNN		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Macro avg	0.92	0.98	0.93	0.94	0.97	0.94	0.78	0.96	0.80	0.94	0.97	0.95	0.90	0.98	0.91
Weighted avg	1.00	1.00	1.00	1.00	1.00	1.00	0.98	0.98	0.98	0.99	0.99	0.99	0.99	0.99	0.99
Accuracy		100%			100%			98%			99%			99%	

Table 16
Data distribution for zero-day attack Teardrop.

ZD Teardrop	Cluster-ID 0		
	Before	Outliers only	All data
Count	290	94	384
Mean	1.455	7.630	2.966
Max	5.764	9.116	9.116
d-min	1.09	6,98	x
Score Sil	0.651	x	0.428
Online learning	YES		

Table 17
Online learning for zero-day attack Teardrop.

Model	DT	RF	KNN	NB	Our CNN
Normal score	1.0	1.0	0.99	0.98	0.99
Online score	0.997	0.955	0.74	0.989	0.966
Label F1	0.996	0.955	0.739	0.849	0.81
Model to kept	YES	YES	NO	YES	YES

- **Online learning and continuous adaptation**: pose challenges regarding the ideal frequency of model updates to capture new threat patterns effectively without overwhelming the data. Implementing mechanisms for regular and real-time updates is crucial for maintaining the model's relevance.
- **Managing the increasing volume of data:** Addressing the continuous surge in data flow without compromising the model's accuracy requires considering mechanisms for data selection, processing, storage and optimizing the framework's deployment.
- **Anomaly detection:** Determining the threshold for segregating outliers with appropriate clusters and handling the potentially high number of newly generated zero-day classes demands thorough exploration.
- **Other case studies:** Exploring additional potential scenarios, such as the presence of outliers within the training data or validation data of a new zero-day class residing in distinct clusters, would be intriguing.
- **Updating the KB:** Incorporating new zero-day classes into the KB enhances detection but raises concerns about maintaining reliability amid continuous updates.

  Hence, all these discussions underscore the need for future research for improved detection of zero-day attacks in general and specifically for our framework for detecting these attacks.

8. Conclusion

In this paper, we present a novel and effective approach for detecting both known attacks and zero-day exploits within network flows. By employing data collection techniques, data preprocessing, and supervised learning through 1D-CNN, we successfully identify these threats. The astute utilization of supervised algorithms such as DT, RF, KNN and NB, coupled with a boosting method, ensures an enhanced detection rate with a high detection accuracy of 98 – 100%. Furthermore, the incorporation of the unsupervised K-Means algorithm to extract flow patterns and generate a correlation table provides cross-validation between supervised and unsupervised methodologies. Our framework identifies anomalies by pinpointing outliers within clusters and correlation tables, thereby offering a dual validation mechanism for new data. Moreover, the validation of zero-day attacks using online learning demonstrates the capability of our solution to identify previously unlearned attack classes. This validation is substantiated by test cases on IBM and NSL-KDD datasets, where malicious activities such as DOS attacks and compromise indicators were identified, including the teardrop class within the NSL-KDD dataset. Looking ahead, we contemplate a more exhaustive exploration of diverse data sources to enhance detection and encompass a broader spectrum of attacks and botnets. In addition, the implementation of our proposition within an industrial environment holds promise for automating correlation rule creation, zero-day vulnerability detection, and real-time attack mitigation.

CRediT authorship contribution statement

Almamy Touré: Writing – original draft, Visualization, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Youcef Imine:** Writing – review & editing, Visualization, Validation, Supervision, Project administration, Funding acquisition, Formal analysis, Conceptualization. **Alexis Semnont:** Investigation, Formal analysis, Data curation, Conceptualization. **Thierry Delot:** Writing – review & editing, Supervision, Project administration, Conceptualization. **Antoine Gallais:** Writing – review & editing, Validation, Project administration, Methodology, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Almamy TOURE reports financial support was provided by IBM Global Business Consulting Services. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table 18
Comparing approaches: Attack coverage, models, accuracy.

Approach	Evaluation dataset	MITRE TACTIC covered	Learning model	Model accuracy
Our framework	IBM + NSL-KDD	TA0001 - TA0002 - TA0003 - TA0008 - TA0010 - TA0011 - TA0040 - TA0042	Hybrid (Supervised + Unsupervised + Online learning)	98.4% IBM and 96.6% NSL-KDD
Agathe et al. [35]	MAWI + UCSD	TA0011	Statistical measures	78.6% MAWI and 85.7% UCSD
Zhou et al. [53]	CIC-AWS-2018	TA0001 - TA0002 - TA0006 - TA0011 - TA0040	Supervised	96–100%
Hindy et al. [34]	CICIDS2017 + NSL-KDD	TA0001 - TA0004 - TA0040 - TA0043	Unsupervised (Auto-Encoder)	90.01–99.67% CICIDS and 92.96% NSL-KDD
Pu et al. [37]	NSL-KDD	TA0001 - TA0004 - TA0040 - TA0043	Unsupervised	52–99%
Comar et al. [38]	Private data from Internet service provider	TA0002	Hybrid (Supervised + Probabilistic Profiling)	88, 54%
Shamsul et al. [39]	Data from CA Technologies VET Zoo	TA0002	Hybrid (Semi-Supervised)	100%

Data availability

The data that has been used is confidential.

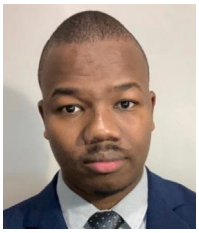
Acknowledgments

This research is part of a collaborative project "CIFRE n° 2020/1084" between IBM Services Center France, the Polytechnic University of Hauts-de-France and the National Center for Scientific Research (CNRS).

References

[1] IBM security report: Data breach cost 2023, https://www.ibm.com/fr-fr/reports/data-breach.
[2] C. Zhang, G. Wang, S. Wang, D. Zhan, M. Yin, Cross-domain network attack detection enabled by heterogeneous transfer learning, Comput. Netw. 227 (2023) 109692.
[3] W. Tounsi, H. Rais, A survey on technical threat intelligence in the age of sophisticated cyber attacks, Comput. Secur. 72 (2018) 212–233.
[4] B. Guembe, A. Azeta, S. Misra, V.C. Osamor, L. Fernandez-San, V. Pospelova, The emerging threat of ai-driven cyber attacks: A review, Appl. Artif. Intell. 36 (1) (2022) 2037254.
[5] B. Bayoğlu, I. Soğukpınar, Graph based signature classes for detecting polymorphic worms via content analysis, Comput. Netw. 56 (2) (2012) 832–844.
[6] P. Garcia-Teodoro, J. Diaz-Verdejo, J. Tapiador, R. Salazar-Hernandez, Automatic generation of HTTP intrusion signatures by selective identification of anomalies, Comput. Secur. 55 (2015) 159–174.
[7] B. Tran, J. Li, A. Madry, Spectral signatures in backdoor attacks, in: S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 31, Curran Associates, Inc., 2018.
[8] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: A survey, ACM Comput. Surv. 41 (3) (2009).
[9] R. Samrin, D. Vasumathi, Review on anomaly based network intrusion detection system, in: 2017 International Conference on Electrical, Electronics, Communication, Computer, and Optimization Techniques, ICEECCOT, 2017, pp. 141–147.
[10] M.A. Jabbar, S. Samreen, Intelligent network intrusion detection using alternating decision trees, in: 2016 International Conference on Circuits, Controls, Communications and Computing (I4C), 2016, pp. 1–6.
[11] B.S. Sharmila, R. Nagapadma, Intrusion detection system using Naive Bayes algorithm, in: IEEE International WIE Conference on Electrical and Computer Engineering (WIECON-ECE), 2019.
[12] H. Zhao, Y. Feng, H. Koide, K. Sakurai, An ANN based sequential detection method for balancing performance indicators of IDS, in: 7th International Symposium on Computing and Networking, CANDAR, 2019, pp. 239–244.
[13] R. Vinayakumar, M. Alazab, K.P. Soman, P. Poornachandran, A. Al-Nemrat, S. Venkatraman, Deep learning approach for intelligent intrusion detection system, IEEE Access 7 (2019) 41525–41550.
[14] T. Zoppi, A. Ceccarelli, A. Bondavalli, Unsupervised algorithms to detect zero-day attacks: Strategy and application, IEEE Access 9 (2021) 90603–90615.
[15] A. Touré, Y. Imine, T. Delot, A. Gallais, A. Semnont, R. Giraudo, Automated and improved detection of cyber attacks via an industrial ids probe, in: N. Meyer, A. Grocholewska-Czuryło (Eds.), ICT Systems Security and Privacy Protection, Springer Nature Switzerland, Cham, 2024, pp. 191–206.
[16] H. Kılıç, N.S. Katal, A.A. Selçuk, Evasion techniques efficiency over the IPS/IDS technology, in: 4th International Conference on Computer Science and Engineering, UBMK, 2019, pp. 542–547.
[17] IBM security topic: What is a zero-day exploit?, www.ibm.com/topics/zero-day.
[18] Z. Zhang, L. Liu, X. Lu, Z. Yan, H. Li, Encrypted network traffic classification: A data driven approach, in: 2020 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom), IEEE, 2020, pp. 706–712.
[19] S. Sen, O. Spatscheck, D. Wang, Accurate, scalable in-network identification of p2p traffic using application signatures, in: Proceedings of the 13th International Conference on World Wide Web, 2004, pp. 512–521.
[20] T. Karagiannis, K. Papagiannaki, M. Faloutsos, BLINC: multilevel traffic classification in the dark, in: Proceedings of the 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, 2005, pp. 229–240.
[21] G. Min, M. Ould-Khaoua, A performance model for wormhole-switched interconnection networks under self-similar traffic, IEEE Trans. Comput. 53 (5) (2004) 601–613.
[22] O. Salman, I.H. Elhajj, A. Kayssi, A. Chehab, A review on machine learning-based approaches for Internet traffic classification, Ann. Telecommun. 75 (11) (2020) 673–710.
[23] L. Koc, A.D. Carswell, Network intrusion detection using a HNB binary classifier, in: 17th UKSim-AMSS International Conference on Modelling and Simulation (UKSim), 2015, pp. 81–85.
[24] P. Owezarski, Investigating adversarial attacks against random forest-based network attack detection systems, in: NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium, 2023, pp. 1–6.
[25] M. Macas, C. Wu, W. Fuertes, A survey on deep learning for cybersecurity: Progress, challenges, and opportunities, Comput. Netw. 212 (2022) 109032.
[26] S. Sivamohan, S. Sridhar, S. Krishnaveni, An effective recurrent neural network (RNN) based intrusion detection via bi-directional long short-term memory, in: International Conference on Intelligent Technologies, CONIT, 2021.
[27] W. Wang, Y. Sheng, J. Wang, X. Zeng, X. Ye, Y. Huang, M. Zhu, HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection, IEEE Access 6 (2018) 1792–1806.
[28] M. Azizjon, A. Jumabek, W. Kim, 1D CNN based network intrusion detection with normalization on imbalanced data, in: International Conference on Artificial Intelligence in Information and Communication, ICAIIC, 2020, pp. 218–224.
[29] M. Mohammadi, B. Raahemi, A. Akbari, B. Nassersharif, Class dependent feature transformation for intrusion detection systems, in: 2011 19th Iranian Conference on Electrical Engineering, 2011, p. 1.
[30] A. Aleroud, G. Karabatis, Toward zero-day attack identification using linear data transformation techniques, in: 2013 IEEE 7th International Conference on Software Security and Reliability, 2013, pp. 159–168.
[31] D. Jin, J. Xie, S. Chen, J. Yang, X. Liu, W. Wang, Zero-day traffic identification using one-dimension convolutional neural networks and auto encoder machine, in: 2020 IFIP Networking Conference (Networking), 2020, pp. 559–563.
[32] K. Roshan, A. Zafar, An optimized auto-encoder based approach for detecting zero-day cyber-attacks in computer network, in: 2021 5th International Conference on Information Systems and Computer Networks, ISCON, 2021, pp. 1–6.
[33] B. Kızıltaş, E. Gül, Network anomaly detection with convolutional neural network based auto encoders, in: 2020 28th Signal Processing and Communications Applications Conference, SIU, 2020, pp. 1–4.
[34] H. Hindy, R. Atkinson, C. Tachtatzis, J.-N. Colin, E. Bayne, X. Bellekens, Utilising deep learning techniques for effective zero-day attack detection, Electronics 9 (10) (2020).

[35] A. Blaise, M. Bouet, V. Conan, S. Secci, Detection of zero-day attacks: An unsupervised port-based approach, Comput. Netw. 180 (2020) 107391.
[36] G. Pu, L. Wang, J. Shen, F. Dong, A hybrid unsupervised clustering-based anomaly detection method, Tsinghua Sci. Technol. 26 (2) (2020) 146–153.
[37] G. Pu, L. Wang, J. Shen, F. Dong, A hybrid unsupervised clustering-based anomaly detection method, Tsinghua Sci. Technol. 26 (2) (2021) 146–153.
[38] P.M. Comar, L. Liu, S. Saha, P.-N. Tan, A. Nucci, Combining supervised and unsupervised learning for zero-day malware detection, in: 2013 Proceedings IEEE INFOCOM, 2013, pp. 2022–2030.
[39] S. Huda, S. Miah, M. Mehedi Hassan, R. Islam, J. Yearwood, M. Alrubaian, A. Almogren, Defending unknown attacks on cyber-physical systems by semi-supervised approach and available unlabeled data, Inform. Sci. 379 (2017) 211–228.
[40] IBM security topic: Qradar architecture and deployment, www.ibm.com/docs/en/qsip/7.5?topic=overview-qradar-events-flows.
[41] S.K. Ray, S. Susan, Performance evaluation using online machine learning packages for streaming data, in: 2022 International Conference on Computer Communication and Informatics, ICCCI, 2022, pp. 1–6.
[42] A. Ed-Daoudy, K. Maalmi, Real-time machine learning for early detection of heart disease using big data approach, in: 2019 International Conference on Wireless Technologies, Embedded and Intelligent Systems, WITS, 2019, pp. 1–5.
[43] T. Talaei Khoei, H. Ould Slimane, N. Kaabouch, Deep learning: systematic review, models, challenges, and research directions, Neural Comput. Appl. 35 (31) (2023) 23103–23124.
[44] S. Jeong, J.-H. You, J.W.-K. Hong, Design and implementation of virtual TAP for SDN-based OpenStack networking, in: 2019 IFIP/IEEE Symposium on Integrated Network and Service Management, IM, 2019, pp. 233–241.
[45] W. Zhenqi, W. Xinyu, NetFlow based intrusion detection system, in: 2008 International Conference on MultiMedia and Information Technology, 2008, pp. 825–828.
[46] S.U. Rehman, W.-C. Song, M. Kang, Network-wide traffic visibility in OF@TEIN SDN testbed using sFlow, in: The 16th Asia-Pacific Network Operations and Management Symposium, 2014, pp. 1–6.
[47] N. Chen, R.E. Johnson, JFlow: Practical refactorings for flow-based parallelism, in: 2013 28th IEEE/ACM International Conference on Automated Software Engineering, ASE, 2013, pp. 202–212.
[48] A. Shahraki, M. Abbasi, A. Taherkordi, A.D. Jurcut, A comparative study on online machine learning techniques for network traffic streams analysis, Comput. Netw. 207 (2022) 108836.
[49] S.S. Sekharan, K. Kandasamy, Profiling SIEM tools and correlation engines for security analytics, in: International Conference on Wireless Communications, Signal Processing and Networking (WiSPNET), 2017, pp. 717–721.
[50] MITRE ATT&CK framework, 2015-2024, www.attack.mitre.org/.
[51] N. Solekha, Analysis of NSL-KDD dataset for classification of attacks based on intrusion detection system using binary logistics and multinomial logistics, Semin. Nas. Off. Stat. 2022 (1) (2022) 507–520.
[52] M. Tavallaee, E. Bagheri, W. Lu, A.A. Ghorbani, A detailed analysis of the KDD CUP 99 data set, in: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009, pp. 1–6.
[53] Q. Zhou, D. Pezaros, Evaluation of machine learning classifiers for zero-day intrusion detection–An analysis on CIC-AWS-2018 dataset, 2019, arXiv preprint arXiv:1905.03685.



Almamy Touré is a Ph.D. student in cybersecurity at the Université Polytechnique des Hauts-de-France while serving as a specialist in security incident detection and response at IBM CIC France. He holds a Master's degree in Computer Science, specializing in cybersecurity from the Université Polytechnique Hauts-de-France (UPHF) in 2019. His research focuses on collecting and analyzing communication flows to detect cyberattacks.